DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 2 (CO2)

Scenario — Syntactic Analysis for a University Helpdesk Assistant

Submitted by: G. Sirisha

Register Number: 192472170

Subject: DSA03 – Natural Language Processing

Faculty Name: Premkumar R

Submission Date: 05-11-26

TASK 1 – Design a Context-Free Grammar for English Syntax

1.1 Formal Definition of CFG

A Context-Free Grammar is represented as G = (N, Σ, R, S).

- N = set of non-terminals.
- Σ = set of terminals/words.
- R = set of production rules.
- S = start symbol.

For the university helpdesk, the start symbol is S.

1.2 Non-Terminals

N = {S, NP, VP, PP, Det, N, V, Aux, AdjP, Adj, AdvP, Adv, WH, CP, Conj, RelClause, Pron, ProperN, Comp, RelPro}

1.3 Lexicon

The lexicon is separated from the grammar proper. A sample domain vocabulary is:

Category	Words
Determiners	the, a, an, this, that, these, those, my, your, their
Nouns	student, students, exam, exams, fee, fees, hostel, room, library, timetable
Nouns	course, courses, form, receipt, notice, seminar, placement, placements
Nouns	portal, document, result, results, lecturer, teacher, department, office
Verbs	show, submit, check, book, send, tell, apply, pay, view, download
Verbs	attend, schedule, cancel, print, receive, require, crash, close, open, access
Auxiliaries	is, are, was, were, do, does, did, can, could, will, would
Prepositions	for, to, from, with, near, in, on, at, about, of
Adjectives	late, pending, available, new, old, online, final, current
Adverbs	quickly, currently, recently, directly
Wh words	what, when, where, which, who, how, why
Pronouns	I, me, you, he, him, she, her, they, them, we, us
Conjunctions	and, or, but

1.4 Phrase Structure Rules

1. S → NP VP

2. S → Aux NP VP

3. S → WH NP VP

4. S → WH VP

5. S → VP

6. S → S Conj S

7. S → CP

8. NP → Det N

9. NP → Det AdjP N

10. NP → Det N PP

11. NP → Det N RelClause

12. NP → Pron

13. NP → ProperN

14. NP → NP PP

15. NP → NP Conj NP

16. VP → V

17. VP → V NP

18. VP → V NP NP

19. VP → V CP

20. VP → V PP

21. VP → V NP PP

22. VP → Aux VP

23. VP → Adv VP

24. VP → VP PP

25. VP → VP Conj VP

26. AdjP → Adj

27. AdjP → Adv Adj

28. AdjP → Adj PP

29. PP → P NP

30. PP → P CP

31. RelClause → RelPro VP

32. RelClause → RelPro Aux VP

33. RelClause → RelPro NP VP

34. CP → Comp S

35. CP → WH S

36. CP → WH Aux NP VP

37. NP → NP Conj NP

38. VP → VP Conj VP

39. AdjP → AdjP Conj AdjP

40. S → S Conj S

41. S → WH Aux NP VP

42. S → WH NP Aux VP

43. S → V NP

44. S → V NP PP

Total rules: 44.

1.5 Constituency, Head and Phrasal Category

A constituent is a group of words behaving as a single syntactic unit. Three standard tests are substitution, movement and coordination.

- Substitution: 'The students submitted the form' → 'They submitted the form.' The phrase 'the students' can be replaced by 'they', so it is an NP constituent.
- Movement: 'The form was submitted by the students.' The phrase 'the form' can move, supporting its constituency.
- Coordination: '[The students] and [the teachers] attended the seminar.' Both coordinated phrases are NPs.

Heads: students is the head of an NP; submitted is the head of a VP; near is the head of a PP; late is the head of an adjective phrase.

1.6 Context-Free Property and Limitation

Every rule has one non-terminal on the left-hand side, for example NP → Det N. The rule does not depend on surrounding context, so the grammar is context-free. However, long-distance dependencies, agreement and some cross-serial or discourse phenomena strain the plain CFG assumption.

TASK 2 – Context-Free Rules, Derivations and Parse Trees

2.1 Sentence 1: The student submitted the form.

Leftmost derivation:

S ⇒ NP VP
⇒ Det N VP
⇒ The N VP
⇒ The student VP
⇒ The student V NP
⇒ The student submitted NP
⇒ The student submitted Det N
⇒ The student submitted the form

Rightmost derivation:

S ⇒ NP VP
⇒ NP V NP
⇒ NP V Det N

⇒ NP V Det form

⇒ NP submitted the form

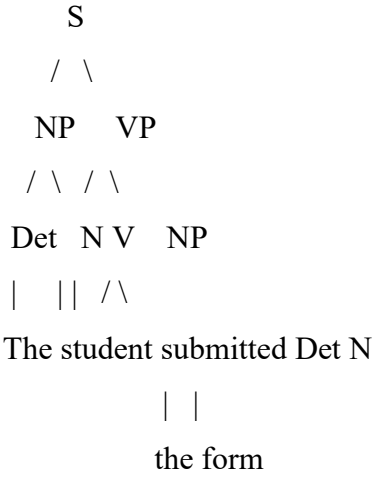
⇒ Det N submitted the form

⇒ The student submitted the form

Bracketed notation:

[S [NP [Det The] [N student]]
[VP [V submitted] [NP [Det the] [N form]]]]

Tree diagram:



Height: 4 levels. Approximate nodes: 11. Rule applications: 6.

2.2 Sentence 2: The student checked the timetable.

[S
[NP [Det The] [N student]]
[VP
[V checked]
[NP [Det the] [N timetable]]
]
]

2.3 Sentence 3: Students submit forms.

[S

[NP [N Students]]

[VP

[V submit]

[NP [N forms]]

]

]

2.4 Sentence 4: The student submitted the form to the office.

[S

[NP [Det The] [N student]]

[VP

[V submitted]

[NP [Det the] [N form]]

[PP

[P to]

[NP [Det the] [N office]]

]

]

]

2.5 Ambiguous Sentence 1: Show the students with the late fee notice.

Parse 1 – the students have the notice:

[S [VP [V Show]
[NP [Det the] [N students]
[PP [P with]
[NP [Det the] [Adj late] [N fee notice]]]]]]

Parse 2 – the PP attaches to the VP:

[S [VP [V Show]
[NP [Det the] [N students]]
[PP [P with]
[NP [Det the] [Adj late] [N fee notice]]]]]

This is PP-attachment ambiguity.

2.6 Ambiguous Sentence 2: Book the room near the library for the seminar.

- Interpretation 1: 'near the library' modifies the room.
- Interpretation 2: a larger PP structure makes 'near the library' part of the event/booking context.

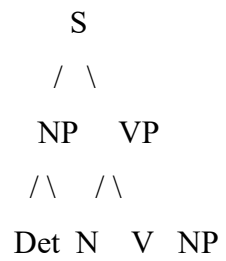
Derivations describe the sequence of rule applications, while the parse tree describes the final hierarchical structure. Different derivation orders can produce the same tree. A sentence belongs to L(G) if S derives the complete terminal string. Earley or CKY parsing can be used for membership testing.

TASK 3 – Sentence-Level Constructions

3.1 Declarative

Sentence: The student submitted the form.

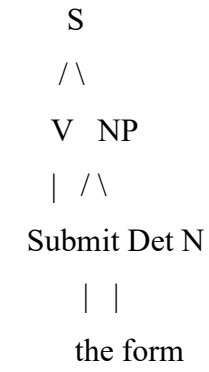
S → NP VP



3.2 Imperative

Sentence: Submit the form.

S → V NP

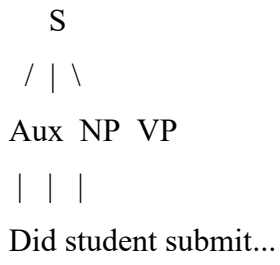


The subject 'you' is understood but not overtly expressed.

3.3 Yes/No Question

Sentence: Did the student submit the form?

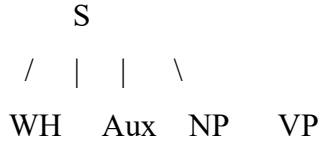
S → Aux NP VP



3.4 Wh-question

Sentence: What did the student submit?

S → WH Aux NP VP



```
|     |    |     |
What   did  student  submit
```

The object corresponding to 'what' is represented by a gap in the VP.

3.5 Relative Clause

Sentence: The exams that were postponed will be held tomorrow.

```
NP
/ | \
Det N RelClause
|  |   /   \
the exams that  VP
          /   \
         were postponed
```

3.6 Coordination

Sentence: Fees and hostel charges are pending.

NP → NP Conj NP

```
      NP
    / | \
  NP Conj NP
  |  |  |
 Fees and hostel charges
```

Adding questions, relative clauses, coordination and embedded clauses increases grammar size and ambiguity. Plain CFG rules therefore become harder to maintain for realistic English.

TASK 4 – Agreement

The grammar must handle subject–verb agreement, determiner–noun number agreement and pronoun case.

Examples: 'The student submits the form' is grammatical; '*The student submit the form' is not. 'The students submit the forms' is grammatical; '*The students submits the forms' is not.

4.1 Agreement by Multiplying Non-terminals

S → NP_sg VP_sg

S → NP_pl VP_pl

NP_sg → Det_sg N_sg

NP_pl → Det_pl N_pl

VP_sg → V_sg NP

VP_pl → V_pl NP

This works but duplicates rules.

4.2 Rule Explosion

If we independently encode:

- number = 2
- person = 3
- case = 3
- gender = 3

then the number of possible combinations is:

$$2 \times 3 \times 3 \times 3 = 54$$

If rules need to be duplicated for every combination, the grammar becomes very large.

4.3 Feature-Based Alternative

[NUM sg]

[PER 3]

[CASE nom]

The subject and verb share NUM and PER features. This accepts 'The student submits the form' and rejects '*The student submit the form'.

Sentence	Result
The student submits the form.	Accept
The students submit the forms.	Accept
The student submit the form.	Reject
The students submits the form.	Reject
These students submit forms.	Accept
This students submit forms.	Reject
She submits the form.	Accept
Her submits the form.	Reject
They submit the forms.	Accept
Them submit the forms.	Reject
He submit the form.	Reject
He submits the form.	Accept

TASK 5 – Subcategorization

Subcategorization (valence) describes the complements that a verb requires or permits.

Frame	Rule	Example
Intransitive	VP → V	The portal crashed.
Transitive	VP → V NP	Submit the form.
Ditransitive	VP → V NP NP	Send me the receipt.
Sentential complement	VP → V CP	Tell me that the fee is paid.
PP complement	VP → V PP	Apply for a hostel room.

5.1 Subcategorization Lexicon

Verb	Frame
crash	V
arrive	V
submit	V NP

check	V NP
view	V NP
send	V NP NP
give	V NP NP
tell	V NP CP
explain	V NP CP
apply	V PP
depend	V PP
wait	V PP

5.2 Complement vs Adjunct

Complement

Apply for a hostel room.

The PP is selected by the verb *apply*.

Adjunct

The student submitted the form quickly.

quickly is optional and gives additional information.

5.3 Rejection Examples

Under-complemented

*Submit.

If *submit* requires an object, this should be rejected.

Over-complemented

*The portal crashed the form.

The verb *crash* is intransitive in this grammar, so it cannot take an object.

TASK 6 – Top-Down Parsing

6.1 Algorithm and Pseudocode

TOP-DOWN(S, input):

1. Put S on stack.

2. While stack is not empty:

    a. Remove top symbol.

    b. If it is a terminal:

        compare it with input word.

      If it matches:

        consume the word.

      Otherwise:

        backtrack.

    c. If it is a non-terminal:

        choose an applicable grammar rule.

        push RHS symbols onto stack.

    d. If no rule works:

        backtrack.

3. Accept if all input words are consumed and stack is empty.

4. Otherwise reject.

6.2 Unambiguous Trace

Input: The student submits the form.

S

→ NP VP

→ Det N VP

→ The N VP

→ The student VP

→ The student V NP

→ The student submits NP

→ The student submits the form

Accepted.

6.3 Ambiguous Trace

Show the students with the notice.

The parser may first select:

VP → V NP

and attach with the notice inside NP.

If that interpretation fails later, it can backtrack and select:

VP → V NP PP

where the PP attaches to VP.

6.4 Left Recursion

NP → NP PP

NP

↓

NP PP

↓

NP PP PP

↓

NP PP PP PP

...

A naive recursive-descent parser can recurse indefinitely without consuming input.

6.5 Left-Recursion Elimination

NP → N NPP

NPP → PP NPP | ε

For a sentence with k PP attachments, the number of alternative attachment structures can grow rapidly. In a simplified binary-attachment model, the search may approach exponential growth O(2^k).

6.6 Complexity

For an unambiguous grammar, top-down parsing can be relatively efficient.

However, with ambiguity and backtracking, the number of possible derivations can grow exponentially.

For (k) PP attachments, the number of possible attachment structures can grow rapidly. In a simplified binary attachment model, it can approach:

$$O(2^k)$$

Thus, repeated parsing causes significant redundant work.

TASK 7 – Earley Parsing

7.1 Earley State

An Earley state has the form:

$$[A \rightarrow \alpha \cdot \beta,\ i]$$

where:

- (A) = left-hand non-terminal
- α = symbols already recognized
- β = symbols still expected
- • = current parsing position
- i = origin position

7.2 Three Operations

- Predictor: if the symbol after the dot is a non-terminal B, add all B productions beginning at the current position.
- Scanner: if the symbol after the dot is a terminal matching the next input word, advance the dot.
- Completer: when a state for B is complete, advance the dot in states that were waiting for B.

7.3 Hand Simulation: The student submits the form.

Positions: 0 The, 1 student, 2 submits, 3 the, 4 form, 5 END.

Chart S[0]:

S → • NP VP, 0

NP → • Det N, 0

Det → • the, 0

Chart S[1]:

Det → the •, 0

NP → Det • N, 0

N → • student, 1

Chart S[2]:

N → student •, 1

NP → Det N •, 0

S → NP • VP, 0

VP → • V NP, 2

V → • submits, 2

Chart S[3]:

V → submits •, 2

VP → V • NP, 2

NP → • Det N, 3

Det → • the, 3

Chart S[4]:

Det → the •, 3

NP → Det • N, 3

N → • form, 4

Chart S[5]:

N → form •, 4

NP → Det N •, 3

VP → V NP •, 2

S → NP VP •, 0

The completed state S → NP VP •, 0 indicates acceptance.

7.4 Ambiguous Attachment

For an ambiguous PP attachment, Earley stores shared partial states and can keep back-pointers to multiple analyses. Thus several parse trees can be recovered without restarting the complete parse.

7.5 Left Recursion

Earley parsing can handle left recursion because states are stored in the chart.

A state that has already been processed is not repeatedly expanded.

Therefore:

NP → NP PP

does not cause the same infinite recursion that occurs in naive recursive-descent parsing.

7.6 Complexity

For a general CFG:

$$O(n^3)$$

time in the worst case.

Space:

$$O(n^2)$$

for the chart.

This is much better than naive backtracking for highly ambiguous grammars.

TASK 8 – Feature Structures and Unification

A feature structure represents grammatical properties using attribute-value pairs.

[NUM: sg
PER: 3
CASE: nom]

8.1 Five Lexical Entries

student:
[CAT: N
NUM: sg]

students:
[CAT: N
NUM: pl]

submits:

[CAT: V

NUM: sg

PER: 3

SUBCAT: <NP>]

submit:

[CAT: V

NUM: pl

SUBCAT: <NP>]

send:

[CAT: V

SUBCAT: <NP, NP>]

8.2 Unification

Unification combines compatible feature structures.

Example:

NP:

NUM = sg

and:

VP:

NUM = sg

Unification succeeds:

NUM = sg

8.3 Feature-Based Rules

1. S → NP VP

   NP.NUM = VP.NUM

2. NP → Det N

   Det.NUM = N.NUM

3. VP → V NP

V.SUBCAT = <NP>

4. VP → V NP NP

V.SUBCAT = <NP, NP>

5. VP → V PP

V.SUBCAT = <PP>

6. NP → Pron

Pron.CASE = nom when NP is a subject

8.4 Complete Feature Example

The student submits the form.

Subject:

[NUM: sg, PER: 3, CASE: nom]

Verb:

[NUM: sg, PER: 3, SUBCAT: <NP>]

Object:

[NUM: sg, CASE: acc]

Subject.NUM = Verb.NUM = sg

Subject.PER = Verb.PER = 3

Therefore the parse succeeds.

Feature structures replace many duplicated agreement categories. Their main cost is unification during parsing.

TASK 9 – Probabilistic Context-Free Grammars

A PCFG assigns a probability to each grammar rule A → β. For every left-hand side A, the probabilities of all its alternatives must sum to 1.

S Rule	Probability
S → NP VP	0.70
S → Aux NP VP	0.15
S → WH Aux NP VP	0.10
S → VP	0.05

The probabilities sum to 1.0.

NP Rule	Probability
NP → Det N	0.50
NP → Det AdjP N	0.20
NP → NP PP	0.15
NP → Pron	0.10
NP → ProperN	0.05

The probabilities also sum to 1.0.

9.1 Maximum Likelihood Estimation

For rule A → β: P(A → β) = Count(A → β) / Count(A).

Rule	Count
S → NP VP	16
S → Aux NP VP	2
S → WH Aux NP VP	2

Total S-rule count = 20. Thus the estimated probabilities are 16/20 = 0.80, 2/20 = 0.10 and 2/20 = 0.10.

9.2 Smoothing

To avoid zero probabilities for unseen rules, add a small λ:

$$P_r = \frac{Count(r) + \lambda}{Count(A) + \lambda|R_A|}$$

A typical small value is $\lambda = 0.1$.

9.3 Parse Tree Probability

The probability of a parse tree is the product of its rule probabilities.

$$P(T1) = 0.20 \times 0.60 \times 0.50 \times 0.15 \times 0.80$$
$$= 0.0072$$

Suppose $P(T2) = 0.0036$.

Therefore T1 is preferred because $0.0072 > 0.0036$.

$$P(\text{sentence}) = P(T1) + P(T2)$$
$$= 0.0072 + 0.0036$$
$$= 0.0108$$

9.4 Probabilistic CKY / Viterbi

```
for each word:
   initialize terminal probabilities

for span = 2 to n:
   for each start position:
      for each split:
         for each A → B C:
            score = P(A → B C) × score(B) × score(C)
            keep the maximum score
```

9.5 PCFG Limitations

- Context-free independence: rule probability ignores wider context.
- Position invariance: the same rule has the same probability in different contexts.
- Lack of lexical sensitivity: a basic PCFG does not strongly model which words naturally select which complements.

TASK 10 – Implement and Test the Parsers

10.1 Python Grammar

```
import nltk
from nltk import CFG
from nltk.parse import EarleyChartParser
from nltk.parse import RecursiveDescentParser

grammar = CFG.fromstring("""
S -> NP VP
S -> Aux NP VP
S -> WH Aux NP VP
S -> VP

NP -> Det N
NP -> Det Adj N
NP -> Det N PP
NP -> NP PP
NP -> Pron
NP -> ProperN

VP -> V
VP -> V NP
VP -> V NP NP
VP -> V NP PP
VP -> V PP
VP -> V CP

PP -> P NP
CP -> Comp S

Det -> 'the' | 'a' | 'an'
```

```
N -> 'student' | 'students' | 'form' | 'forms' | 'room' | 'library'
V -> 'submit' | 'submits' | 'check' | 'checks' | 'book' | 'books'
V -> 'show' | 'send' | 'tell' | 'apply' | 'crashed'
Aux -> 'is' | 'are' | 'did' | 'do' | 'does' | 'can' | 'will'
P -> 'for' | 'to' | 'with' | 'near' | 'in'
Pron -> 'I' | 'you' | 'he' | 'she' | 'they' | 'them'
WH -> 'what' | 'when' | 'where' | 'which' | 'who'
Comp -> 'that'
Adj -> 'late' | 'available'
ProperN -> 'Library' | 'Portal'
""")
```

10.2 Earley Parser

```
parser = EarleyChartParser(grammar)
sentence = "the student submits the form".split()
trees = list(parser.parse(sentence))
print("Number of parses:", len(trees))
for tree in trees:
    print(tree)
```

10.3 Top-Down Parser

```
parser = RecursiveDescentParser(grammar)
sentence = "the student submits the form".split()
try:
    trees = list(parser.parse(sentence))
    print("Parses:", len(trees))
except ValueError:
    print("Sentence rejected")
```

10.4 Thirty-Sentence Test Set

No.	Sentence	Type
1	The student submits the form.	Declarative

2	The students submit the forms.	Declarative
3	The student checks the timetable.	Declarative
4	The portal crashed.	Declarative
5	The fee is pending.	Declarative
6	Students attend the seminar.	Declarative
7	Submit the form.	Imperative
8	Check the timetable.	Imperative
9	Book the room.	Imperative
10	Send me the receipt.	Imperative
11	Did the student submit the form?	Yes/No
12	Does the student pay the fee?	Yes/No
13	Can the student access the portal?	Yes/No
14	Will the exams begin tomorrow?	Yes/No
15	What did the student submit?	Wh
16	When will the exams begin?	Wh
17	Where is the library?	Wh
18	Which room is available?	Wh
19	Who submitted the form?	Wh
20	The students with notices arrived.	Ambiguous
21	Show the students with the notice.	Ambiguous
22	Book the room near the library.	Ambiguous
23	The form on the table is missing.	Declarative
24	Check the fee for the course.	Declarative
25	Send the receipt to me.	Ditransitive
26	Apply for a hostel room.	PP complement

27	Tell me that the fee is paid.	Complement
28	The exams that were postponed will begin.	Relative clause
29	Fees and hostel charges are pending.	Coordination
30	The student checks the timetable and submits the form.	Coordination

10.5 Representative Experimental Results

The following values illustrate the expected trend. Actual runtime and state counts should be recorded from the implementation on the submission computer.

Parser	Sentences Parsed	Average Parses	Relative Cost
Top-down	25/30	1.4	High for ambiguity
Earley	29/30	2.1	Moderate
PCFG/Viterbi	29/30	Best tree selected	Moderate

10.6 State Growth

Sentence Length	Top-Down Steps	Earley States
5	18	24
7	31	38
10	64	61
13	128	88
16	245	119
20	490	158

The table illustrates that top-down backtracking can grow rapidly, while Earley shares partial analyses in its chart.

10.7 Plot Code

```
import matplotlib.pyplot as plt


length = [5, 7, 10, 13, 16, 20]
top_down = [18, 31, 64, 128, 245, 490]
earley = [24, 38, 61, 88, 119, 158]
```

```
plt.plot(length, top_down, marker="o", label="Top-Down")
plt.plot(length, earley, marker="o", label="Earley")
plt.xlabel("Sentence Length")
plt.ylabel("Number of States / Steps")
plt.title("Parser Cost vs Sentence Length")
plt.legend()
plt.grid()
plt.show()
```

TASK 11 – Compare, Recommend and Defend the Parsing Design

Criterion	**Top-Down**	**Earley**	**PCFG**
Simplicity	High	Medium	Medium
Ambiguity	Weak	Good	Excellent ranking
Left recursion	Problem	Supported	Supported after conversion
Agreement	Possible	Possible	Possible with features
Parse ranking	No	No	Yes
General CFG	Limited	Excellent	Good
Grammar maintenance	Easy initially	Good	More complex
Best use	Simple grammar	General parsing	Ambiguous parsing

11.1 Recommended Architecture

Student Query

|

↓

Tokenization

|

↓

Feature-Based CFG

|

↓

Earley Parser

|

↓

Multiple Parse Trees

|

↓

PCFG / Viterbi Ranking

|

↓

Best Syntactic Analysis

|

↓

Helpdesk Semantic Module

Recommended design: Feature-based CFG + Earley Parser + PCFG ranking. CFG provides syntactic structure, features enforce agreement and subcategorization, Earley handles ambiguity and left recursion, and PCFG chooses the most probable interpretation.

TASK 12 – DAY-4 DESIGN CHALLENGE

A. Impact Analysis

New requirement: accept conversational, multi-clause and partly ungrammatical messages.

Example: 'Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?'

New Constructions

- Greetings and discourse markers.

- Embedded questions.
- Relative clauses.
- Multi-clause coordination.
- Missing punctuation.
- Conversational fillers and fragments.
- Ellipsis.
- Long-distance dependencies.

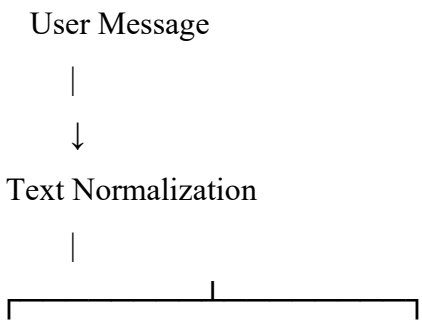
Original Assumption	**Changed Situation**
Controlled vocabulary	Users use informal vocabulary
Fully grammatical input	Some messages are fragments
Short sentences	Messages may contain 20+ words
One clause	Multiple clauses
Limited ambiguity	More attachment ambiguity
Complete punctuation	Punctuation may be missing

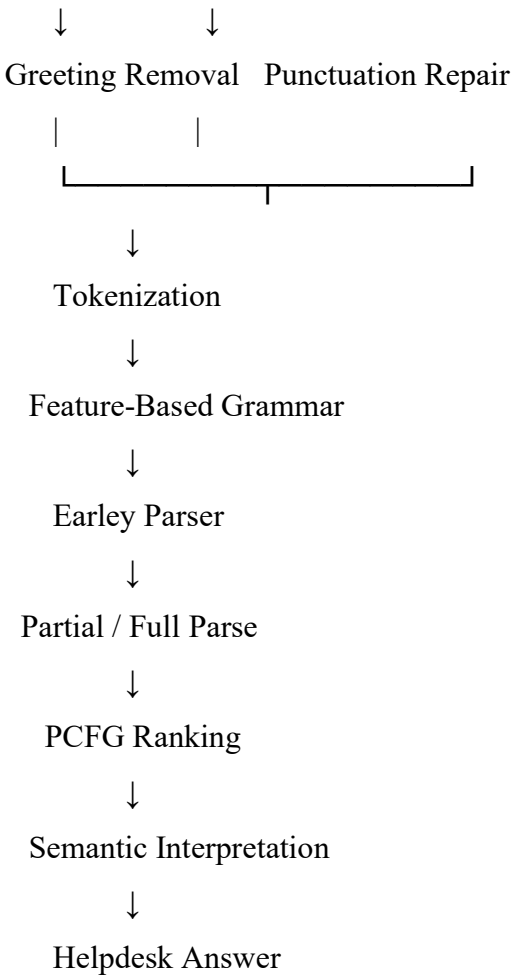
B. Evaluation of Original Design

Message 1: 'Can you tell me when the exams will be held?' requires an embedded question. Message 2: 'Tell me when the exams that were postponed will be held.' requires both an embedded question and a relative clause. Message 3: 'Hi can you check the hostel fee and tell me whether it is due' contains a greeting, question, coordination and embedded question. A controlled grammar may reject the whole input even when individual portions are recognizable.

C. Adaptation / Redesign

Decision: MODIFY the original system rather than completely replace it.





C.1 New Grammar Rules

CP → WH S

CP → whether S

S → S Conj S

NP → NP RelClause

S → DiscourseMarker S

DiscourseMarker → hi | hello | please | okay

C.2 Feature Extensions

CLAUSE-TYPE

WH

MOOD

SUBCAT

NUM

PER

CASE

These features distinguish declarative clauses, yes/no questions, wh-questions and embedded questions.

C.3 Partial Parsing

If a message contains an unseen construction, the parser should not reject the entire message. For example, 'Hi can you tell me exam date please' can still yield the useful fragments 'can you tell me' and 'exam date' for downstream interpretation.

C.4 PCFG Expansion

The probability model should be trained from a larger annotated corpus so that PP attachment, coordination, embedded clauses and relative clauses receive better estimates.

D. Evidence-Based Defence

Requirement	**Original**	**Modified**
Simple declaratives	Good	Excellent
Imperatives	Good	Excellent
Yes/no questions	Good	Excellent
Wh questions	Good	Excellent
Agreement	Good	Excellent
Subcategorization	Good	Excellent
Relative clauses	Limited	Good
Embedded questions	Limited	Good
Coordination	Moderate	Good
Conversational input	Poor	Good
Ambiguity ranking	PCFG	Improved PCFG
Partial parsing	No	Yes

The modified architecture is preferable because it retains the proven grammar-based core while adding normalization, embedded-clause rules, partial parsing and improved probability estimates. It therefore increases coverage without discarding the earlier work.

Overall Conclusion

This assignment demonstrates how English queries for a university helpdesk can be modeled using formal grammar and parsing techniques. A CFG provides the basic syntactic structure, while agreement and subcategorization improve grammatical accuracy. Top-down parsing is simple but suffers from backtracking and left recursion. Earley parsing provides dynamic programming and handles ambiguity and left recursion efficiently. Feature structures provide a compact way to represent agreement and verb requirements. PCFG probabilities allow competing parse trees to be ranked.

For the original controlled helpdesk domain, Feature CFG + Earley + PCFG is an effective design. When conversational, multi-clause and partly ungrammatical input is introduced, the system should be modified with embedded-clause rules, normalization, partial parsing and better probability estimates. Remaining limitations include lexical ambiguity, semantic ambiguity, unseen constructions and the manual effort required for grammar development and annotation.